

Actividad de Investigación y Práctica: Estructuras de Datos Avanzadas y APIs con ASP.NET Core

Parte 1: Investigación teórica

1. Estructura de Datos Eficientes

Árboles Binarios de Búsqueda (ABB): Son una estructura de datos jerárquica en la que cada nodo puede tener como máximo dos hijos: un hijo izquierdo y un hijo derecho. Su propiedad principal establece que todos los valores del subárbol izquierdo deben ser menores que el valor almacenado en el nodo actual, mientras que todos los valores del subárbol derecho deben ser mayores. Esta organización permite realizar operaciones de búsqueda, inserción y eliminación de manera eficiente, ya que en cada comparación se descarta aproximadamente la mitad del árbol. En condiciones ideales, cuando el árbol se mantiene balanceado, estas operaciones tienen una complejidad promedio de $O(\log n)$; sin embargo, en el peor caso, cuando los datos se insertan en orden secuencial y el árbol se degenera hasta parecerse a una lista enlazada, la complejidad puede llegar a $O(n)$.

Árboles AVL: Son árboles binarios de búsqueda auto balanceados, diseñados para mantener una altura adecuada y evitar que la estructura se deforme. Su principio central es el factor de balanceo, que se calcula como la diferencia entre la altura del subárbol izquierdo y la del subárbol derecho. En un árbol AVL, este valor debe ser -1, 0 o 1 en cada nodo. Cuando una inserción o eliminación rompe esta condición, el árbol se reordena mediante rotaciones simples o dobles, con el fin de recuperar el equilibrio. Gracias a este mecanismo, las operaciones de búsqueda, inserción y eliminación conservan una complejidad de $O(\log n)$, lo que hace a esta estructura especialmente útil en aplicaciones que requieren consultas frecuentes y buen rendimiento.

2. Fundamentos de Web APIs:

API y Modelo Cliente-Servidor: Una API es un conjunto de reglas que permite que dos aplicaciones se comuniquen e intercambien información de forma organizada. En el modelo cliente-servidor, el cliente realiza una solicitud para acceder a un recurso o ejecutar una acción, mientras que el servidor recibe la petición, la procesa y devuelve una respuesta. Este modelo facilita la separación de responsabilidades, ya que el cliente se enfoca en la interfaz o el consumo de datos, y el servidor en la lógica del negocio y el acceso a la información.

Protocolo HTTP: HTTP es el protocolo que hace posible la comunicación entre clientes y servidores en la web. Su funcionamiento se basa en un esquema de petición y respuesta: el cliente envía una solicitud con un método, una dirección y, en algunos casos, datos adicionales; luego, el servidor responde con un código de estado y la información solicitada, normalmente en formatos como JSON. Además, HTTP es un protocolo sin estado, lo que significa que cada petición se procesa de manera independiente.

Verbos HTTP (diferencia, uso e idempotencia): Los verbos HTTP indican la acción que el cliente desea realizar sobre un recurso. Su uso correcto permite diseñar APIs más claras y ordenadas. Un concepto importante es la idempotencia, que significa que repetir una misma petición varias veces produce el mismo efecto en el servidor que ejecutarla una sola vez.

- **GET:** Se utiliza para obtener información de un recurso sin modificar su contenido. Es un método idempotente, porque repetir la misma solicitud no cambia el estado del servidor. Por esta razón, es el verbo más usado para consultas, búsquedas y recuperación de datos en una API.
- **POST:** Se emplea principalmente para crear nuevos recursos o enviar información que será procesada por el servidor. No es idempotente, ya que si la misma petición se ejecuta varias veces, puede generar varios registros o resultados distintos. Es común en formularios, registros de usuarios y envío de datos a una API.
- **PUT:** Se usa para actualizar por completo un recurso existente o crearlo si no existe. Es idempotente, porque repetir la misma petición mantiene el recurso en el mismo estado final. Su utilidad principal está en reemplazar información completa dentro de una API.
- **DELETE:** Su función es eliminar un recurso identificado dentro del servidor. También se considera idempotente, porque después de borrar el recurso, repetir la misma petición no produce un cambio adicional en el estado del sistema. Se utiliza para operaciones de eliminación dentro de servicios web y APIs REST.